

ECE586 - Computer Architecture

Group - 9

Roles and Responsibilities

Member	Area	Responsibilities
Nikhil Swarna	Trace Reader, Types & Main Driver, Instruction Decoder & Statistics	Implemented trace_reader.c/.h: memory image file loading, line-by-line hex parsing, whitespace trimming, 8-digit validation, and bounds-checked word storage. Defined types.h: all shared enums (opcode_t, format_t, category_t, stage_t), instruction_t struct, and project-wide constants (MEM_WORDS, NUM_REGS, PIPE_STAGES). Implemented decoder.c/.h: opcode-indexed metadata table mapping all 18 opcodes to mnemonic, format (R/I), and category; decoder_decode() extracting rs, rt, rd, and sign-extended imm from the 32-bit word; and decoder_mnemonic() for readable opcode labels. Implemented stats.c/.h: stats_t counters for total, arithmetic, logical, memory, and control instructions; stats_count_instruction() dispatch; and stats_print_instruction_counts() report formatting.
Venkat Sai Sumanth Koyada	Without forwarding	Implemented timing_sim.c/.h for the no-forwarding pipeline: timing_t pipeline struct with 5-slot pipe array, clock/stall/hazard/flush counters, and mode flag; dest_reg() and src_regs() helpers identifying producer/consumer register dependencies; produces_for() RAW dependency check; compute_stalls() implementing no-forwarding stall rules (2 cycles for load-use, 1 cycle for all others by distance); shift() pipeline advance; timing_step() driving fetch, stall insertion, and branch flush; timing_drain() draining the last 4 stages; and timing_print_report() formatting CPI, hazard counts, and flush cycles.
Venkata Sriram Kamarajugadda	Functional Simulator	Implemented func_sim.c/.h: machine_t state struct (PC, 32-entry register file, reg_written change log, memory pointer); func_sim_init() zeroing all state; func_sim_step() executing all 18 instructions using signed 32-bit two's complement arithmetic; reg_write() helper enforcing R0 hardwiring; word_index() helper for aligned bounds-checked memory access; effective address computation for LDW/STW;

		and PC update logic for BZ, BEQ, and JR. Implemented main.c: top-level simulation driver that initializes machine state, runs the fetch-decode-execute loop, drives both timing simulators in lockstep, and formats the final report.
Hanisha Produtur	With Forwarding	Implemented timing_sim.c/.h for the forwarding pipeline: timing_t pipeline struct with 5-slot pipe array, clock/stall/hazard/flush counters, and mode flag; dest_reg() and src_regs() helpers identifying producer/consumer register dependencies; produces_for() RAW dependency check; compute_stalls() implementing forwarding rules (load-use 1 stall, all others 0 stalls); shift() pipeline advance; timing_step() driving fetch, stall insertion, and branch flush; timing_drain() draining the last 4 stages; and timing_print_report() formatting CPI, hazard counts, and flush cycles.